

Optimization Methods for Data Science
Final Project
SVM+Bonus Part

Gabriele Cabibbo 2196717
Giorgio Taramanni 1961217

September 2025

1 Introduction

A **Support Vector Machine** (SVM) is a machine learning model mainly used for linear and nonlinear classification tasks, but can be generalized for regression and clustering applications.

It learns a set of weights λ and biases b as the optimal solution of the dual (non)linear SVM problem

$$\begin{cases} \min_{\lambda} & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j K(x_i, x_j) - \sum_{i=1}^n \lambda_i \\ & \sum_{i=1}^n \lambda_i y_i = 0, \\ & 0 \leq \lambda_i \leq C, \quad i = 1, \dots, n \end{cases}$$

And the decision function associated with those parameters is:

$$y(x) = \text{sign} \left(\sum_{i=1}^L \lambda_i y_i K(x_i, x_j) + b \right)$$

in which $K(x_i, x_j)$ is a **kernel** function, that allows us to implicitly map the input into an high dimensional feature space where linear classification can be performed (**kernel trick**), hence it allows us to perform nonlinear classification. Some frequently used kernels include, with p and γ as hyperparameters.:

- Gaussian: $K(x, y) = e^{-\gamma \|x-y\|^2}$
- Polynomial $K(x, y) = (x^T y + 1)^p$

The aforementioned decision function defines the best hyperplane separating the two classes. That is, the hyperplane associated with the largest margin between the classes. Thus, we choose the hyperplane so that the distance from it to the nearest data point (**support vectors**) on each side is maximized.

2 Implementation

In the following implementations, the response variable is encoded as $\{-1, +1\}$ instead of $\{0, 1\}$ and the features have been standardized prior to the training process.

We implemented a training process that takes in input the Hessian matrix $P_{i,j} = y_i y_j K(x_i, x_j)$, the column vector q , the box constraints on the Lagrangian multipliers $0 \leq \alpha_i \leq C \forall i$ and the equality constraint $\sum_i \alpha_i y_i = 0$. Then this quadratic programming problem is solved using the function `solvers.qp` from the `cvxopt` library. In this way, we compute the Lagrangian multipliers α_i associated with the dual problem, we then identify the support vectors as the samples where $\alpha_i > 0$. The bias term is then computed. If any free support vectors ($0 < \alpha_i < C$) exist, the bias is computed as $b = y_i - \sum_j \alpha_j y_j K(x_j, x_i)$. Otherwise, the bias is determined using all support vectors.

We then implemented an algorithm based on the **Sequential Minimal Optimization** (SMO) method. This solves the dual problem of the SVM by optimizing a pair of Lagrangian multipliers α_i, α_j at each step. These α values are chosen using the **Most Violating Pair** technique, which selects the pair that maximizes the error difference $|E_i - E_j|$. Initially, all α are set to 0. The kernels are computed and we pick the indices that do not satisfy the KKT conditions. If such a point exists, then the other point, j , is selected to maximize the error difference. We update the pair (α_i, α_j) within the bounds L and H , which are obtained from the box constraints $0 \leq \alpha_i, \alpha_j \leq C$ and the relationship between the labels y_i and y_j ,

Following each update, the bias parameter b is recomputed as:

$$b_1 = b_{\text{old}} - E_i - y_i(\alpha_i^{\text{new}} - \alpha_i^{\text{old}})K_{ii} - y_j(\alpha_j^{\text{new}} - \alpha_j^{\text{old}})K_{ij}$$

$$b_2 = b_{\text{old}} - E_j - y_i(\alpha_i^{\text{new}} - \alpha_i^{\text{old}})K_{ij} - y_j(\alpha_j^{\text{new}} - \alpha_j^{\text{old}})K_{jj}$$

The final bias value b is chosen as $b = b_1$, $b = b_2$, or their mean $\frac{b_1 + b_2}{2}$, depending on whether α_i or α_j are strictly within the interval $(0, C)$.

Training ends when no α is modified for a certain number of consecutive passes, or once a maximum number of iterations is reached. Similarly to the first method, the **support vectors** are the samples with $\alpha_i > 0$.

We can build a Multiclass SVM with two possible approaches: **One-vs-Rest** and **One-vs-One**. The former consists in training K separate binary classifiers that learn to distinguish one class ("positive") from all the remaining classes ("negative"). When making a prediction for a new data point, the input is fed to all the classifiers and the output is the one corresponding to the classifier with the highest output score. The latter builds $\frac{K(K-1)}{2}$ binary classifiers, one for each pair of classes, and a prediction is obtained by letting each binary classifier vote for one of the classes on which it was trained and choosing the class with the highest number of votes.

3 Training and Validation

A k -fold cross-validation with $k = 5$ folds was performed for both the `solvers.qp` and the SMO fit methods. The hyperparameter grid in Table 1 includes values for the regularization term C , the Gaussian kernel parameter γ , and the Polynomial kernel degree p . These values were generated using the functions `np.arange` and `np.linspace`. This grid was applied to both training methods, evaluating 480 combinations of C with all other kernel parameters to identify the optimal SVM structure.

For the multiclass training, all possible combinations of three classes were created from the five present in the dataset. Both the One-vs-One and One-vs-Rest methods were tested using both the classic (`solvers.qp`) and SMO fit.

This allowed us to identify which class pairings and comparisons produced the best performance for the multiclass SVM. In some cases, smaller grid searches were carried out to find the best parameters for the multiclass setting.

4 Results

4.1 Binary Classification

Figure 1 shows the results of the grid search for models trained using `solver.qp`. We can notice how there are no substantial differences between the best models obtained with Gaussian and Polynomial kernels, meaning that we can get high performances using both kernels. The right panel of the figure shows that SVMs with the Gaussian kernel reach very high values consistently for values of C higher than a certain threshold. In contrast, results obtained with Polynomial kernels appear less sensitive to C , likely because the degree of the polynomial kernel plays a more dominant role in determining the model’s performance than the regularization parameter. This observation is supported by Figure 2, which displays the performance of each kernel type as a line plot. For the Gaussian kernel, once again, the value of C is highly influential on the accuracy: performance remains high for all SVMs, except for $C = 0.05$, regardless of the value of gamma. Conversely, for the Polynomial kernel, performance is almost exclusively influenced by the *degree* of the kernel, while C has a negligible effect.

The histogram in the left panel of Figure 3 compares the performance of the best models obtained for each kernel, suggesting that the SMO training method is more effective for the Gaussian kernel. On the right panel of the figure, the polynomial kernel shows a behavior similar to the previous training method: its performance is exclusively influenced by the degree parameter, with no sensitivity to the regularization parameter C . In contrast, the Gaussian kernel shows a slightly different trend. For low values of C , the accuracy remains high, irrespective of the gamma value. However, as C increases, performance becomes more dependent on the gamma parameter. Figure 4 confirms our conclusions on the relationship between polynomial kernel and C . Meanwhile, the Gaussian kernel shows no discernible pattern, except for the line corresponding to $C = 0.1$, which demonstrates consistently high performance across all gamma values. In Table 2, we can observe that, for both training methods, the accuracy values are very good, showing absence of overfitting and good generalization ability.

4.2 Multiclass Classification

In the multiclass approach, we compared the results obtained for each combination of classes, with both training methods. Table 2 displays main characteristics of the best models for each type of multiclass SVM. For both methods the best combination of classes was (1, 2, 3) but we achieved better results fitting the OvR model using `solvers.qp`, while training the OvO model using SMO resulted in better scores. The parameters of the SVMs used are the same. Accuracies show

valid performance in this case too, with no presence of overfitting.

List of Figures

1	GridSearch results for classic fit.	7
2	Accuracy trend for both kernels with various parameter combinations and C (classic fit)	8
3	GridSearch results for SMO fit.	9
4	Accuracy trend for both kernels with various parameter combinations and C (SMO fit)	10

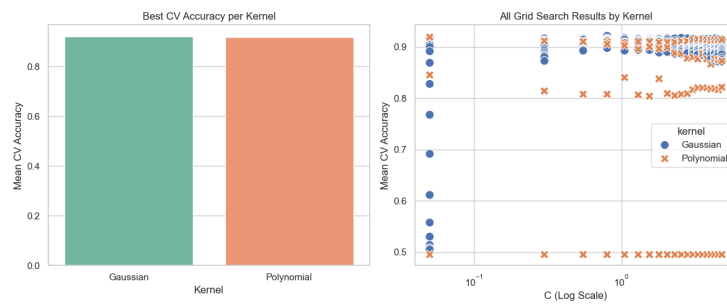


Figure 1: GridSearch results for classic fit.

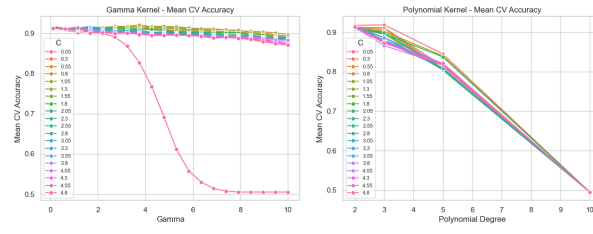


Figure 2: Accuracy trend for both kernels with various parameter combinations and C (classic fit)

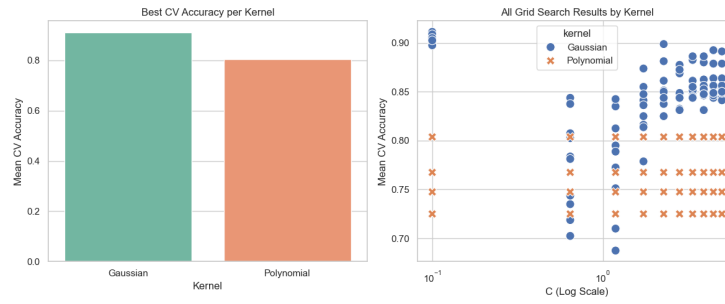


Figure 3: GridSearch results for SMO fit.

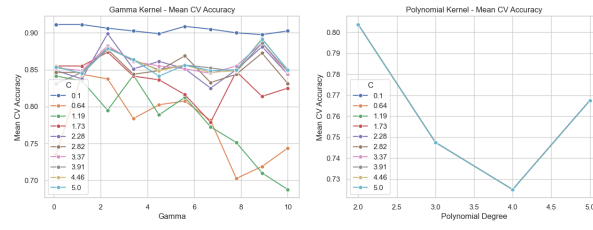


Figure 4: Accuracy trend for both kernels with various parameter combinations and C (SMO fit)

List of Tables

1	Hyperparameter Grid	12
2	Best Models	13

Table 1: Hyperparameter Grid

Hyperparameter	Values in the Grid
Regularization term (C)	0.05, 0.3, 0.55, 0.8, 1.05, 1.3, 1.55, 1.8, 2.05, 2.3, 2.55, 2.8, 3.05, 3.3, 3.55, 3.8, 4.05, 4.3, 4.55, 4.8
Degree (p)	2, 3, 5, 10
γ	0.1, 0.62, 1.14, 1.66, 2.18, 2.71, 3.23, 3.75, 4.27, 4.79, 5.31, 5.83, 6.35, 6.87, 7.39, 7.92, 8.44, 8.96, 9.48, 10.0

Table 2: Best Models

Question	Hyperparameters			Performance		Optimization	
	Kernel	C	γ (or p)	Train Accuracy	Test Accuracy	Iterations	CPU Time
Q2	Gaussian	0.8	$\gamma = 3.75$	95%	91%	10	0.48 s
Q3	Gaussian	0.05	$\gamma = 0.1$	92%	93%	21	0.56 s
Q4 (OvR) classes (1, 2, 3) trained using <code>solvers.qpl</code>	Gaussian	0.05	$\gamma = 0.5$	93.67%	90.33%	36	8.18 s
Q4 (OvO) classes (1, 2, 3) trained using SMO	Gaussian	0.05	$\gamma = 0.5$	93%	89.3%	30	0.68 s